FEU INSTITUTE OF TECHNOLOGY

COLLEGE OF COMPUTER STUDIES AND MULTIMEDIA ARTS

CS0065
(INTELLIGENT SYSTEMS)

TECHNICAL ASSESSMENT

2

RULE – BASED AGENT

Student Name	Fritz Mardaniel L. Sablan
Section:	AN42
Professor:	Ms. Crisola G. Tan
Date:	02/10/2026

I. BACKGROUND INFORMATION

A Rule-Based Agent is a type of intelligent agent that operates by following predefined rules to decide its actions. In this lab, students will work with a vacuum cleaner agent that can clean rooms or move between rooms depending on their state (dirty or clean).

The goal is to understand how agents perceive the environment, act according to rules, and update the environment.

II. TECHNICAL ASSESSMENT

RULE – BASED AGENT

Objective

At the end of this laboratory activity, students are expected to:

1. Explain the concept of rule-based agents.
2. Implement a rule-based agent in Python.
3. Interact with an environment by perceiving and acting based on rules.
4. Observe and analyze the behavior of an agent in a controlled environment.

Materials Needed

- Computer with Python 3.x installed
- Text editor or IDE (e.g., VSCode, PyCharm)
- Optional: Jupyter Notebook for interactive testing

General Instructions

1. This **Technical Assessment (TA4)** must be completed individually.
2. Follow the Task below.
3. Include clear comments in your code to describe its logic.
4. Each program should display the correct input, process, and output.
5. Ensure that your script displays clear and readable outputs.
6. Compile all Python codes and corresponding outputs into one PDF file.
7. Save it as **TA2_lastname.pdf**
   (Example: TA2_Patilano.pdf)
8. Submit the **python file** and **PDF** file.

Laboratory Task

Pre-Lab Preparation

1. Review the Python concepts:
   - Classes and objects
   - If-Else statements

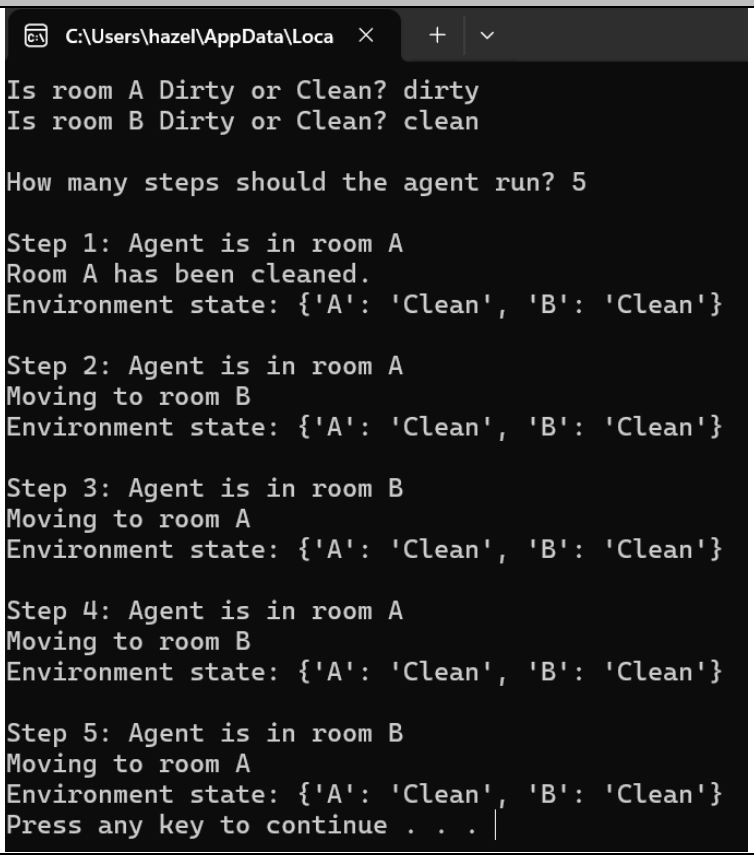
- Loops (for loop)
- Functions / methods

2. Understand the environment:
   - Two rooms: A and B
   - Each room can be Dirty or Clean

Task 1: Set Up the Environment

1. Initialize two rooms using a Python dictionary.
2. Allow the user to set the **initial state** of each room (dirty or clean).
3. Write a function to **check if a room is dirty** and a function to **clean a room**.

Sample Output



Task 2: Implement the Rule-Based Agent

1. Create a class for the agent.
2. Implement the **perceive-and-act method**:
   - Rule 1: If the current room is dirty, clean it.
   - Rule 2: If the current room is clean, move to the other room.
3. Implement a **move method** to switch rooms.

Task 3: Run the Simulation

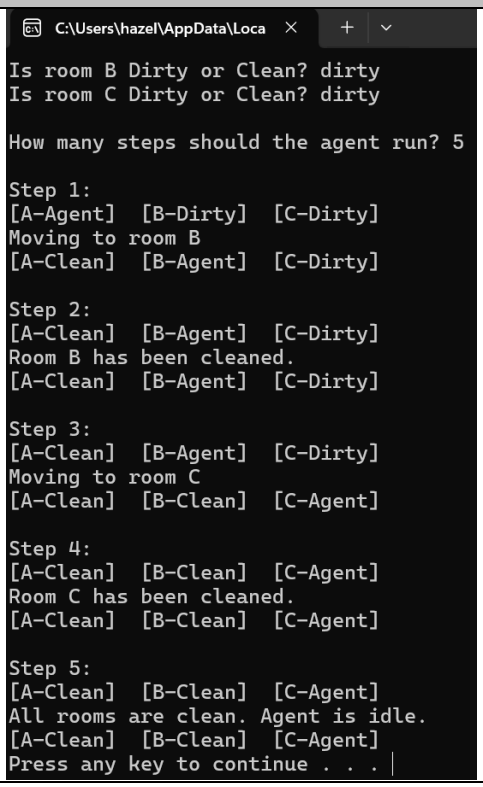
1. Ask the user how many steps the simulation should run.
2. Run the agent in a loop for the specified steps.

3. After each step, print:
    - The agent's current location
    - The action taken (cleaned/moved)
    - Current state of the environment

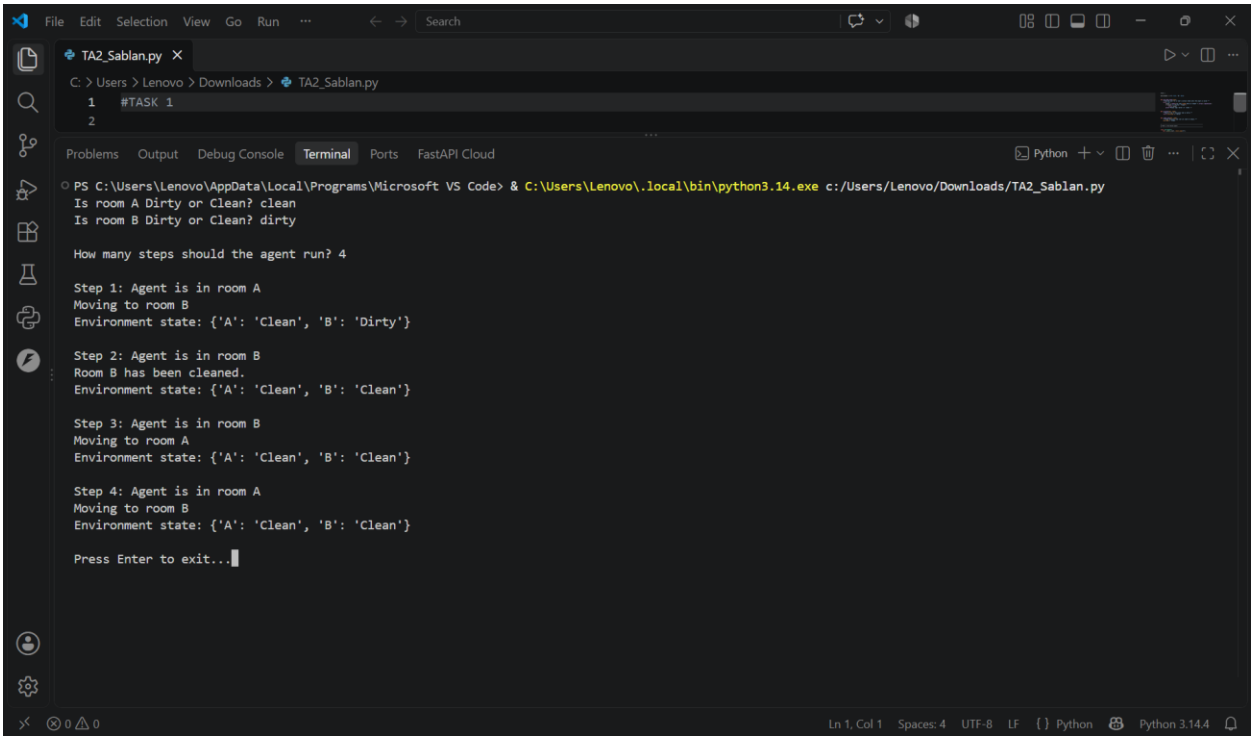
Optional (Bonus Task)

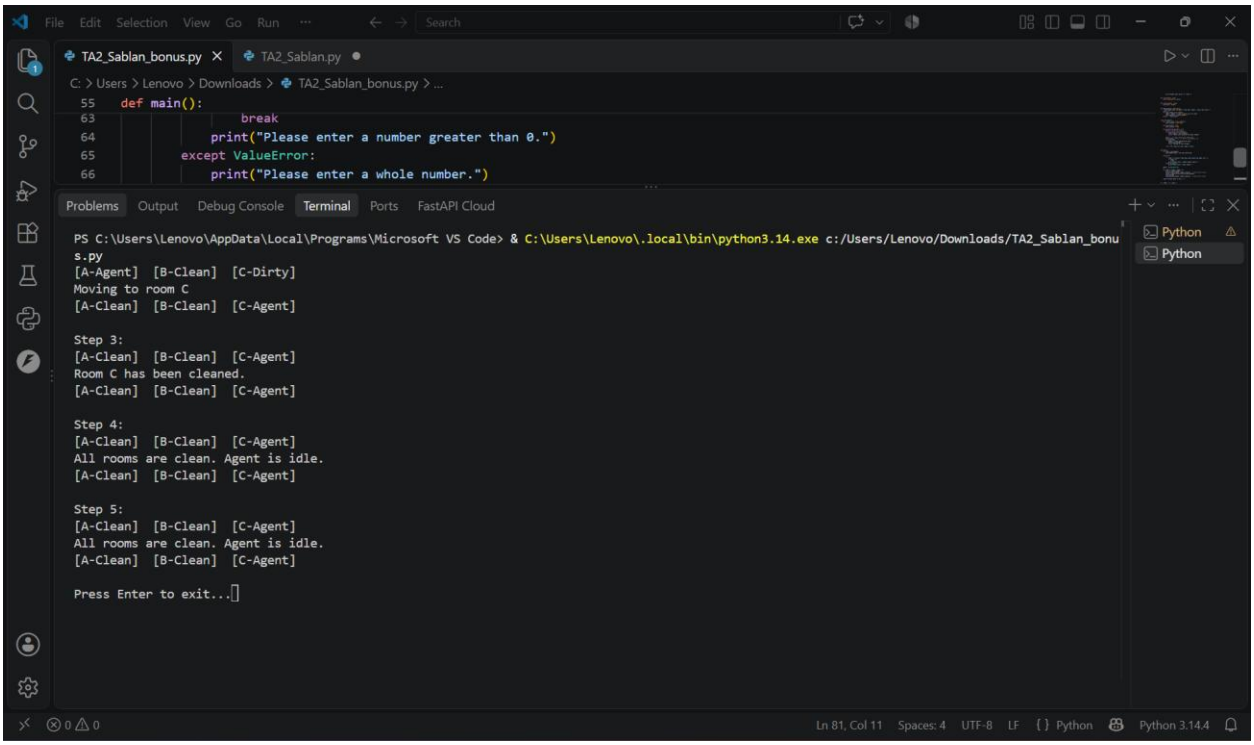
- Add a **third room (C)** and update the agent logic.
- Make the agent **randomly choose which dirty room to clean next**.
- Visualize the agent moving between rooms using a **text-based grid**.

Sample Output (Bonus Task)



Your Code - Laboratory Tasks ⇩


File Edit Selection View Go Run
Search
TA2_Sablan.py
C: > Users > Lenovo > Downloads > TA2_Sablan.py
1 #TASK 1
2
Problems Output Debug Console Terminal Ports FastAPI Cloud
Python
PS C:\Users\Lenovo\AppData\Local\Programs\Microsoft VS Code> & C:\Users\Lenovo\.local\bin\python3.14.exe c:/Users/Lenovo/Downloads/TA2_Sablan.py
Is room A Dirty or Clean? clean
Is room B Dirty or Clean? dirty
How many steps should the agent run? 4
Step 1: Agent is in room A
Moving to room B
Environment state: {'A': 'Clean', 'B': 'Dirty'}
Step 2: Agent is in room B
Room B has been cleaned.
Environment state: {'A': 'Clean', 'B': 'Clean'}
Step 3: Agent is in room B
Moving to room A
Environment state: {'A': 'Clean', 'B': 'Clean'}
Step 4: Agent is in room A
Moving to room B
Environment state: {'A': 'Clean', 'B': 'Clean'}
Press Enter to exit...
Ln 1, Col 1 Spaces: 4 UTF-8 LF { } Python Python 3.14.4


File Edit Selection View Go Run
Search
TA2_Sablan_bonus.py
TA2_Sablan.py
C: > Users > Lenovo > Downloads > TA2_Sablan_bonus.py > ...
55 def main():
63 break
64 print("Please enter a number greater than 0.")
65 except ValueError:
66 print("Please enter a whole number.")
Problems Output Debug Console Terminal Ports FastAPI Cloud
Python
Python
PS C:\Users\Lenovo\AppData\Local\Programs\Microsoft VS Code> & C:\Users\Lenovo\.local\bin\python3.14.exe c:/Users/Lenovo/Downloads/TA2_Sablan_bonus.py
[A-Agent] [B-Clean] [C-Dirty]
Moving to room C
[A-Clean] [B-Clean] [C-Agent]
Step 3:
[A-Clean] [B-Clean] [C-Agent]
Room C has been cleaned.
[A-Clean] [B-Clean] [C-Agent]
Step 4:
[A-Clean] [B-Clean] [C-Agent]
All rooms are clean. Agent is idle.
[A-Clean] [B-Clean] [C-Agent]
Step 5:
[A-Clean] [B-Clean] [C-Agent]
All rooms are clean. Agent is idle.
[A-Clean] [B-Clean] [C-Agent]
Press Enter to exit...
Ln 81, Col 11 Spaces: 4 UTF-8 LF { } Python Python 3.14.4

#TASK 1

```
environment = {"A": None, "B": None}

def ask_room_state(room):
    """Ask the user for a room's initial state until the input is valid."""
    while True:
        answer = input(f"Is room {room} Dirty or Clean? ").strip().capitalize()
        if answer in ("Dirty", "Clean"):
            return answer
        print("Please type 'dirty' or 'clean'.")

def is_dirty(env, room):
    """Return True if the given room is dirty."""
    return env[room] == "Dirty"

def clean_room(env, room):
    """Clean the given room (set its state to Clean)."""
    env[room] = "Clean"

#TASK 2

class VacuumAgent:
    def __init__(self, start_room="A"):
        self.location = start_room

    def move(self):
        """Switch to the other room."""
        self.location = "B" if self.location == "A" else "A"

    def perceive_and_act(self, env):
        """Perceive the current room and act using the rules."""
        if is_dirty(env, self.location):
            #Rule 1 - if dirty, clean it
            clean_room(env, self.location)
            return f"Room {self.location} has been cleaned."
        else:
            #Rule 2 - if clean, move to the other room
            self.move()
            return f"Moving to room {self.location}"

#TASK 3

def main():
    for room in environment:
        environment[room] = ask_room_state(room)
```

```
    #steps
    while True:
        try:
            steps = int(input("\nHow many steps should the agent run? "))
            if steps > 0:
                break
            print("Please enter a number greater than 0.")
        except ValueError:
            print("Please enter a whole number.")

    agent = VacuumAgent("A")

    for step in range(1, steps + 1):
        print(f"\nStep {step}: Agent is in room {agent.location}")
        action = agent.perceive_and_act(environment)  # perceive + act
        print(action)
        print(f"Environment state: {environment}")

    input("\nPress Enter to exit...")

if __name__ == "__main__":
    main()
```

```
#Bonus

import random

environment = {"A": None, "B": None, "C": None}

def ask_room_state(room):
    """Ask the user for a room's initial state until the input is valid."""
    while True:
        answer = input(f"Is room {room} Dirty or Clean? ").strip().capitalize()
        if answer in ("Dirty", "Clean"):
            return answer
        print("Please type 'dirty' or 'clean'.")

def is_dirty(env, room):
    return env[room] == "Dirty"
```

```
def clean_room(env, room):
    env[room] = "Clean"

def show_grid(env, agent_room):
    """Text-based grid: the agent's room shows 'Agent', others show state."""
    cells = []
    for room, state in env.items():
        label = "Agent" if room == agent_room else state
        cells.append(f"[{room}-{label}]")
    print("  ".join(cells))

class VacuumAgent:
    def __init__(self, start_room="A"):
        self.location = start_room

    def move_to(self, room):
        self.location = room

    def perceive_and_act(self, env):
        #Rule 1 - if dirty, clean it
        if is_dirty(env, self.location):
            clean_room(env, self.location)
            return f"Room {self.location} has been cleaned."

        #Rule 2 - if clean, move to the other room
        dirty_rooms = [r for r in env if is_dirty(env, r)]
        if dirty_rooms:
            target = random.choice(dirty_rooms)
            self.move_to(target)
            return f"Moving to room {target}"

        return "All rooms are clean. Agent is idle."

def main():
    for room in environment:
        environment[room] = ask_room_state(room)

    while True:
        try:
            steps = int(input("\nHow many steps should the agent run? "))
            if steps > 0:
                break
            print("Please enter a number greater than 0.")
```

```
        except ValueError:
            print("Please enter a whole number.")

    agent = VacuumAgent("A")

    for step in range(1, steps + 1):
        print(f"\nStep {step}:")
        show_grid(environment, agent.location)   # state before acting
        action = agent.perceive_and_act(environment)
        print(action)
        show_grid(environment, agent.location)   # state after acting

    input("\nPress Enter to exit...")

if __name__ == "__main__":
    main()
```

Task 4: Analysis Questions

After running the simulation, answer the following:

1. What rules does the agent follow?
2. How does the agent decide when to clean and when to move?
3. What happens if both rooms are already clean at the start?
4. How would you modify the agent to handle **more than two rooms**?

Answer the Questions ⇩

1. Rule 1 says that if the current room is dirty, clean it. Rule 2 says that if the current room is clean, move to the other room. Rule 3 says that if all rooms are clean, stay idle.
2. Each step, the agent perceives the state of the room it is in. If it sees that the room is dirty, it cleans. If it sees clean, it moves.
3. It moves back and forth between A and B on every step until the simulation ends.
4. Store the rooms in a list or dictionary, so you can add as many as you like. Then replace the switch to the other room move with a move to a chosen target. The target could be the next room in order, a random dirty room, or the nearest dirty room.